

Lecture 5: The 12-Ball & 13-Ball Problem and Decision Tree Analysis

*Instructor: Sourav Chakraborty, Sanjukta Roy**Scribe: Ashutosh, Saikat*

Problem Statement (12 Ball Problem)

There are 12 balls. Among these, 11 balls are identical in weight and 1 ball is different. The different ball may be either **heavier** or **lighter**.

Goal:

- Identify which ball is different
- Determine whether it is heavier or lighter
- Use the minimum number of weighings on a balance scale

Why This is an Algorithmic Problem

Each weighing is a comparison. The algorithm must decide *what to weigh* based on previous outcomes until the answer is uniquely determined.

Hence, this is a **comparison-based algorithm**, and such algorithms are analyzed using a **decision tree**.

Decision Tree

A decision tree is a diagram shaped like an upside-down tree that shows choices and outcomes.

Every decision tree has three main parts:

Nodes

- Internal nodes represent comparisons (weighings)
- Leaf nodes represent final answers

Branches

Branches represent outcomes:

$<, =, >$

Leaves

- Each leaf corresponds to exactly one valid input configuration
- Reaching a leaf means the algorithm terminates

Decision Tree in Balance-Scale Problems

Each weighing gives 3 possible outcomes, so the algorithm can be modeled as a decision tree:

- Each internal node = one weighing
- Each node has three branches ($<, =, >$)
- Each leaf = a unique conclusion (which ball is odd and heavy/light)

Counting the Number of Possible Outcomes

We must distinguish all possible inputs:

Odd ball choices = 12

Heavy or light $\rightarrow$ 2 possibilities

$$\text{Total outcomes} = 12 \times 2 = 24$$

So, the decision tree must have at least 24 leaves.

Lower Bound on Number of Weighings

A decision tree of height h with 3 outcomes per node has at most 3^h leaves.

$$3^h \geq 24$$

$$h \geq \log_3 24$$

$$h \approx 2.89 \Rightarrow h \geq 3$$

At least 3 weighings are necessary.

Important Observation

Decision tree height gives a **lower bound**, not a guarantee. Even if the lower bound is 3, it does not mean a valid algorithm using 3 weighings exists.

What Goes Wrong in 3 Weighings

Step 1

After the first weighing:

- Either balance occurs
- Or inequality occurs, but we still do not know:
 - Which ball is odd
 - Whether it is heavy or light

Step 2

After the second weighing, we must both:

- Identify the odd ball
- Determine heavy vs light

But a balance scale only gives relative weight.

Step 3

After the third weighing, multiple configurations still produce identical results:

$$(A \text{ heavy}), \quad (B \text{ light}), \quad (C \text{ heavy})$$

These are indistinguishable. Hence 3 weighings are insufficient.

No algorithm using only 3 weighings can solve the 12-ball problem

Minimum Number of Weighings

Minimum weighings = 4

Algorithm (Using 4 Weighings)

Step 1: First Weighing

Weigh:

- Left pan: (B_1, B_2, B_3, B_4)
- Right pan: (B_5, B_6, B_7, B_8)

Case 1.1: Balance (=)

- Then $(B_1) - (B_8)$ are all normal
- The odd ball is among $(B_9, B_{10}, B_{11}, B_{12})$

→ Go to Step 2A

Case 1.2: Not equal ($<$) or ($>$)

- The odd ball is among $(B_1) - (B_8)$
- We now know relative information (which side is heavier)

→ Go to Step 2B

Step 2A: (If Step 1 Balanced)

Weigh:

- Left pan: (B_9, B_{10})
- Right pan: (B_{11}, B_{12})

Case 2A.1: Balance ($=$)

- Impossible (since one ball must be different)

Case 2A.2: Not equal

- We now know which pair contains the odd ball
- And whether it is heavy or light relative to the other

→ Go to Step 3A

Step 3A: (Odd ball in a known pair)

Weigh:

- (B_9) vs (B_{10}) (or (B_{11}) vs (B_{12}))

- If one side is heavier $\rightarrow$ that ball is the odd one
- Direction tells heavy or light

Conclusion: Odd ball identified

Step 2B: (If Step 1 Unbalanced)

Weigh:

- Left pan: (B_1, B_2, B_5)
- Right pan: (B_3, B_9, B_{10})

(Note: (B_9, B_{10}) are known normal balls)

$\rightarrow$ Go to Step 3B

Step 3B: Narrow Down to One Ball

Weigh:

- Two suspect balls against each other
- If equal $\rightarrow$ third ball is odd
- If unequal $\rightarrow$ heavier or lighter is identified

Conclusion: Odd ball identified

Correctness

After 3 weighings, at most two cases may remain. One extra weighing separates them.

$$\text{Max weighings} = 3 + 1 = 4$$

There is no execution path longer than 4 edges in the decision tree.

Algorithm is correct and optimal

Problem Statement (13 Ball Problem)

We are given the following:

- There are 13 balls.
- Exactly one ball is different.
- The odd ball may be either heavier or lighter.
- We are allowed to use a balance scale.
- Each weighing has three possible outcomes:
 - Left side is heavier,
 - Right side is heavier,
 - Both sides balance.

Goal: Identify the odd ball and determine whether it is heavier or lighter.

How many possibilities exist?

- Choosing the odd ball: 13 choices.
- Each ball can be either heavy or light.

Hence, the total number of possible configurations is:

$$13 \times 2 = 26$$

Thus, any algorithm must be able to distinguish between **26 distinct cases**. This requirement is non-negotiable.

Information from weighings

Each weighing has 3 possible outcomes. Therefore:

1 weighing $\rightarrow$ 3 cases,

2 weighings $\rightarrow 3^2 = 9$ cases,

3 weighings $\rightarrow 3^3 = 27$ cases.

Hence, we must have:

$$3^k \geq 26.$$

Since:

$$3^3 = 27 \geq 26,$$

we obtain the lower bound:

$$k \geq \log_3(26).$$

Important: This only shows that fewer than three weighings are impossible. It does *not* guarantee that three weighings are sufficient.

Why a decision tree does not guarantee a solution

Although a decision tree of height 3 has 27 leaves, not all of these outcomes are logically achievable.

- Some outcome sequences contradict physical constraints.
- Outcomes cannot be assigned arbitrarily.

Thus, a decision tree alone does not guarantee a valid algorithm.

Structure of the first weighing

In the first weighing, both pans must contain the same number of balls.

Let:

- x balls be placed on the left pan,
- x balls be placed on the right pan,
- $13 - 2x$ balls remain off the scale.

This structure is unavoidable.

The balanced case

If the scale balances, then:

- The odd ball is not on the scale.
- It lies among the $13 - 2x$ balls.

Each of these balls may be heavy or light, so the number of possibilities is:

$$2(13 - 2x).$$

For example:

- If $x = 4$: remaining balls = 5, possibilities = 10.
- If $x = 5$: remaining balls = 3, possibilities = 6.

Even in the best case, at least 6 possibilities remain after one weighing.

Information remaining

If we aim to finish in 3 weighings total:

- One weighing is already used.
- Only two weighings remain.

Two weighings can distinguish at most:

$$3^2 = 9 \text{ cases.}$$

Thus, after the first weighing, every branch must contain at most 9 possibilities.

The unbalanced case

If the scale is unbalanced:

- The odd ball lies among the $2x$ balls on the scale.
- The direction of tilt gives partial information.
- However, both the identity of the ball and whether it is heavy or light are still unknown.

For reasonable values of x , this branch contains at least 10 possible configurations.

This corresponds to the statement in the notes:

“If $x \leq 4$ make $>$; at least 10 configurations.”

The contradiction

No matter how x is chosen:

- Either the balanced case, or
- the unbalanced case

leaves at least 10 possibilities after the first weighing.

But two remaining weighings can distinguish only:

$$3^2 = 9 < 10.$$

This leads to a contradiction.

Final conclusion

- Three weighings allow at most 27 theoretical outcomes.
- Physical constraints force at least 10 possibilities after the first weighing.
- Two remaining weighings are insufficient to resolve them.

Therefore, three weighings are impossible for both 12 Ball and 13 Ball Problem
--

Minimum number of weighings required = 4
--

Theorem. *Let A be any comparison-based sorting algorithm. Fix any constant α such that $0 < \alpha < 1$ (for example $\alpha = 0.99$). Then there exists a constant $c > 0$ such that for possible input of n distinct elements, algorithm A performs at least $n \log n$ comparisons. Hence, A requires $\Omega(n \log n)$ comparisons on almost all inputs.*

Proof. Any comparison-based sorting algorithm can be represented as a binary decision tree. Each internal node corresponds to one comparison between two input elements, and each leaf corresponds to the final sorted order produced by the algorithm.

Since the input consists of n distinct elements, there are $n!$ possible input permutations. To correctly sort every possible input, the decision tree must have at least $n!$ leaves.

Let k be a nonnegative integer. A binary decision tree can have at most 2^k leaves at depth at most k . Therefore, the number of input permutations that can be sorted using at most k comparisons is at most 2^k .

Assume that the algorithm sorts at least $\alpha \cdot n!$ permutations using at most k comparisons.

Then the number of leaves reachable within depth k must satisfy

$$2^k \geq \alpha \cdot n!.$$

Taking logarithm base 2 on both sides,

$$k \geq \log_2(\alpha \cdot n!) = \log_2(\alpha) + \log_2(n!).$$

Using Stirling's approximation,

$$\log_2(n!) = n \log_2 n - \Theta(n).$$

Substituting into the inequality gives

$$k \geq \log_2(\alpha) + n \log_2 n - \Theta(n).$$

Since $\log_2(\alpha)$ is a constant (because α is fixed), the dominant term is $n \log_2 n$. Hence, for sufficiently large n ,

$$k = \Omega(n \log n).$$

Thus, at least $\alpha \cdot n!$ input permutations require $\Omega(n \log n)$ comparisons, proving the theorem. □